

Open in app ↗



Data Science Colle... · Following

 Member-only story

Running Language models locally: A Framework and UI Comparison

14 min read · May 29, 2025



Prateek Bhatnagar

 Listen Share More

Unless you are living under a rock, you would have heard the explosion of large language models and its evolution from NNs → RNNs → CNNs → Transformers → GPTs. Alongside the development of these colossal models for General purpose AI (GPAI), there has been significant progress and increasing interest in their more compact counterparts: Small Language Models (SLMs). We'll elaborate on the benefits of having these SLMs in the era where FAANG is other hand competing to create even higher spec and parameter models.



Photo by [Andrea De Santis](#) on [Unsplash](#)

SLMs? Why?

Before the widespread accessibility of easy-to-use local deployment frameworks for SLMs, running any significant language model often required substantial infrastructure. While smaller, task-specific models could be deployed locally, the kind of versatile language understanding and generation we associate with modern LLMs was largely confined to powerful servers and cloud platforms. The classic trade-off between model capability and the resources required to run it was a significant barrier for many local applications. This gave rise to SLMs.

SLMs are smaller language generative models with generally only a few billion parameters (e.g. 1–2 billion parameters). One could classify even a 7–13b model as SLM. After Meta's release of LLaMA (2023) — a family of 7B–65B-parameter, the community rapidly developed compact open models (Phi-2, Qwen-1B, TinyLlama, etc.) and optimized runtimes. These allow running LLMs on ordinary laptops (CPU-only or with modest GPUs) without cloud dependencies. To enable this many frameworks, libraries, and open-source contributions have emerged, making it increasingly feasible to experiment with and deploy these models locally. We cover some of this frameworks with codes in this article. The review and remarks below are my personal take basis research and ease of use. It does not involve any branding /promoting element.

Some of the straight solid reasons for SLMs :

- **On-Device AI:** Make it happen locally on your laptop and your mobile
- **Privacy-Preserving Applications:** Building applications locally that process sensitive text data locally without sending it to the cloud
- **Offline Functionality:** Enabling applications to utilize language models in environments without internet connectivity
- **Custom AI :** Creating personalized AI assistants, fine tuned models, knowledge distillation, architecture modifications etc. are just some of the possibilities that open up with these SLMs for wider community who may not have access to high spec computing

Lets Dive into it!

Brief technical primer

- **Quantization** : Quantization is a technique used to reduce the size and computational requirements of large language models (LLMs) by representing their weights with fewer bits (e.g., 4-bit or 8-bit instead of 16-bit or 32-bit). This makes models smaller, faster, and more efficient, enabling them to run on devices with limited hardware resources, such as laptops or consumer GPUs. But this comes at the cost of some accuracy. So the trade off here has to be managed carefully. The GGUF format files discussed below are quantized versions.
- **Distillation** : Training a smaller SLM to mimic the behavior of a larger, more powerful model. This allows the SLM to achieve comparable performance on specific tasks with a significantly smaller footprint. Example could be DistilBERT which has significantly lesser parameters than original BERT of 110 million or the deepseek's distilled Qwen or Llama 7b models
- **Parameters** : They are internal variables (like weights and biases in neural networks) that a model learns during training to make predictions. The number and arrangement of these parameters are like knobs that directly affect the model's capacity, size, and performance. Examples are like over a trillion parameters of chatgpt4 or 70 billion parameters of a Llama model
- **Tokenization** : The process of breaking text into smaller pieces (tokens), often sub-words, before feeding into a language model
- **Context Window (Context Length)** : The number of tokens a model can “see” or consider in a single prompt
- **GGUF Format** : GPT-Generated Unified Format is a quantized model format optimized for local inference (especially with llama.cpp).
- **Instruct versions** are Instruction-tuned to follow user instructions (e.g., “Summarize this”, “Write a poem”), Responds to direct prompts with useful outputs and often limited to single prompt use, Example: LLaMA2-Instruct . **Chat versions** are Chat-optimized for conversations and some memory for context, example: Mistral-7B-Chat . **Code versions** are Code-focused models Trained or fine-tuned specifically for programming tasks — code generation, debugging, explanation. Example: CodeLLaMA , DeepSeek-Coder . **Text versions** are Text completion just trained to complete text, useful for general NLP tasks like language modeling, summarization, translation. Often used as a base for further tuning.

- **FlashAttention** is an algorithm that reorders the attention computation and leverages classical techniques like tiling and recomputation to significantly speed up attention mechanisms in transformers. It reduces memory usage from quadratic to linear in sequence length, enabling faster and more memory-efficient training and inference

Models considered in this notebook :

1. Phi-2 (2.7B parameters)
2. Qwen2 (1.5B)
3. TinyLlama (1.1B)

Local specs for testing below code snippets was very modest with no GPU. Thinkpad Gen 12 , i7 Ultra , 32GB Ram. As you move to larger ones — GPUs will be needed unless you are ok to be patient with super slow inference time

1. Hugging Face and Transformers

Arguably one of the biggest names in the world of language models, Hugging face is driving forward open-source ML and AI development. Having more than a million open source models, it is the default place to go as a start.

Hugging face library has the most direct way to run an SLM via the Hugging Face Transformers library. For example, one can use `AutoModelForCausalLM` and `AutoTokenizer` to load a model and generate text in Python. But first lets download our models locally using `huggingface_hub` . Note this can take several minutes to may be an hour depending on your internet connection. Example for TinyLlama , it took ~2 mins.

Download models

```
import pandas as pd
import numpy as np
from transformers import AutoModelForCausalLM, AutoTokenizer
from huggingface_hub import snapshot_download
import time

# Define the models and local paths
models = {
    "microsoft/phi-2": "C:\\Users\\Mylaptop\\Desktop\\Projects\\LLMs\\phi-2",
```



```

"Qwen/Qwen-1.8B": "C:\\Users\\MyLaptop\\Desktop\\Projects\\LLMs\\qwen-1b"
"TinyLlama/TinyLlama-1.1B-Chat-v1.0": "C:\\Users\\MyLaptop\\Desktop\\Projec
}

# Download each model
for model_name, save_path in models.items():
    print(f"Downloading {model_name} to {save_path}...")
    snapshot_download(repo_id=model_name, local_dir=save_path)
    print(f"Download completed for {model_name}!\n")

print("All models downloaded successfully!")

```

A downloaded model files will look something like this

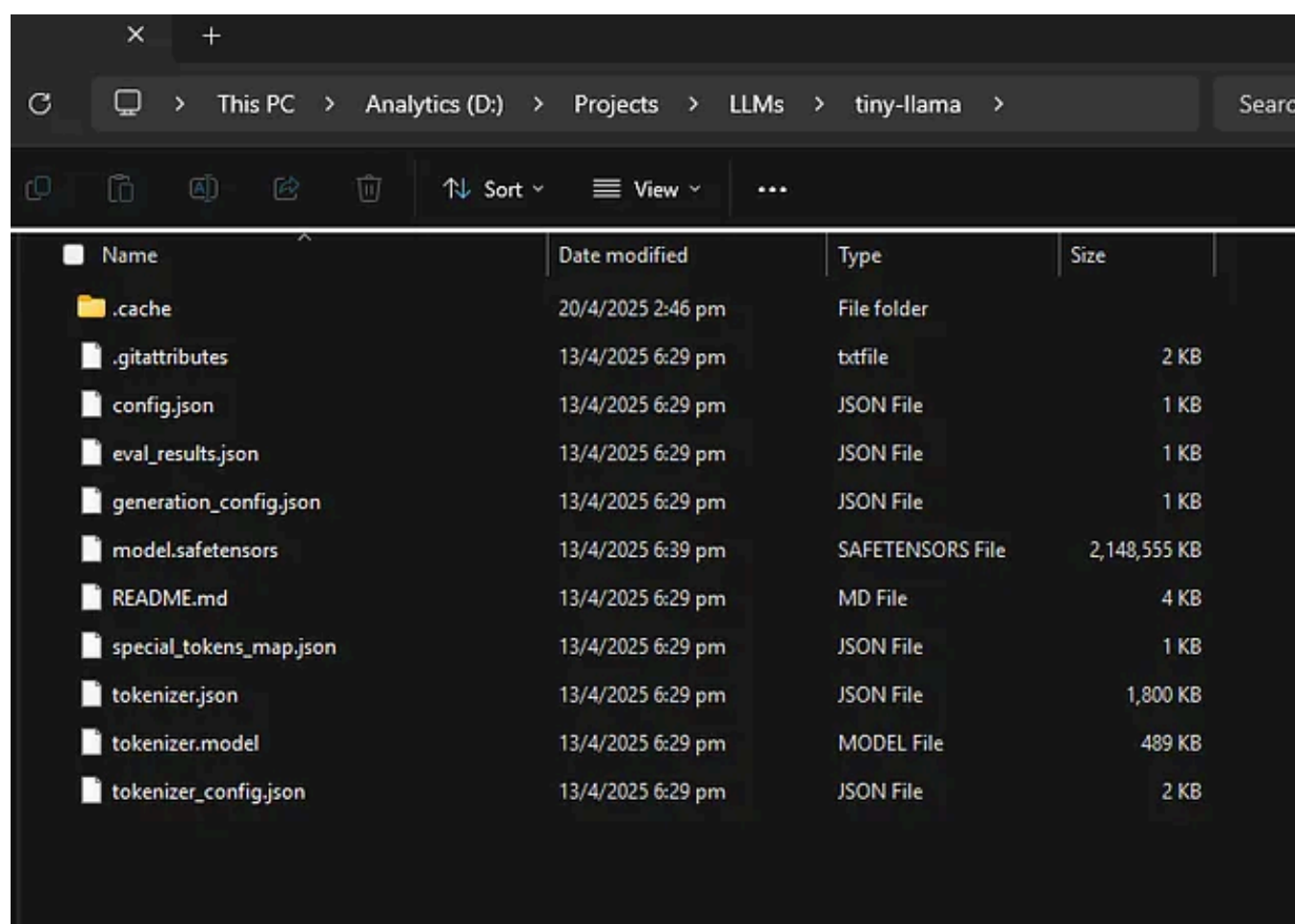


Image by Author

To seek the model of your choice, search it on `huggingface_hub` and get the `repo_id`. While Transformers supports hundreds of models (Phi-2, Qwen, TinyLlama, etc.) directly from Hugging Face but it loads full-precision weights (32-bit by default) and must often as you go for bigger models, will rely on GPU for speed. We see the

impact of this as we compare to other GGUF based formats and C/C++ based libraries.

Chat with models

Lets write a simple function to prompt our models

```
def generate_text(model_llm , prompt, max_length=50, temperature=0.7, num_return_sequences=1):
    """
    Generate text using the loaded model.

    Args:
        prompt (str): The input prompt for the model.
        max_length (int): Maximum length of the generated text.
        temperature (float): Sampling temperature for creativity.
        num_return_sequences (int): Number of sequences to generate.
        do_sample (bool): Whether to use sampling for generation.

    Returns:
        str: The generated text.
    """
    # Load the tokenizer and model
    tokenizer = AutoTokenizer.from_pretrained(model_llm, trust_remote_code=True)
    model = AutoModelForCausalLM.from_pretrained(model_llm, trust_remote_code=True)

    # Tokenize the input prompt
    inputs = tokenizer(prompt, return_tensors="pt")

    # Generate text
    output = model.generate(
        inputs["input_ids"],
        max_length=max_length,
        num_return_sequences=num_return_sequences,
        do_sample=do_sample,
        temperature=temperature
    )

    # Decode and return the generated text
    return tokenizer.decode(output[0], skip_special_tokens=True)
```

Now lets prompt the three models and compare the inference time. Note that there are many parameters that user can tweak but i have only the few key ones above.

```
# Example HF
user_prompt = "Which city is eiffel tower located in?"

t0 = time.time()
generated_text = generate_text(model_llm=phi2,prompt = user_prompt,  temperatur
print("Generated Text by Phi2:")
print(generated_text)
t1 = time.time()
print(f"\nTime taken: { t1-t0:.2f} seconds")

t0 = time.time()
generated_text = generate_text(model_llm=qwen2,prompt = user_prompt,  temperatur
print("Generated Text by Qwen:")
print(generated_text)
t1 = time.time()
print(f"\nTime taken: {t1-t0:.2f} seconds")

t0 = time.time()
generated_text = generate_text(model_llm=tinyllama,prompt = user_prompt, temper
print("Generated Text by TinyLlama:")
print(generated_text)
t1 = time.time()
print(f"\nTime taken: {t1-t0:.2f} seconds")
```

From the below output, you can see the SLMs fetching some extra training data tokens along and not giving a simple answer. This is despite it being the full 32 bit. This is a limitation of the SLMs but there is a wide choice there which may suit the user specific needs. The models we are considered here are amongst the smallest SLMs. TinyLlama was fastest with ~9 seconds where as Qwen and Phi2 with ~12 seconds


```
Loading checkpoint shards: 100%|██████████| 2/2 [00:03<00:00, 1.94s/it]
Generated Text by Phi2:
Which city is eiffel tower located?
A) London
B) Paris
C) Berlin
D) Rome
Answer: B) Paris.

Time taken: 12.28 seconds
Loading checkpoint shards: 100%|██████████| 2/2 [00:04<00:00, 2.19s/it]
Generated Text by Qwen:
Which city is eiffel tower located? Paris or New York? The Eiffel Tower is located in Paris, France.

Time taken: 11.83 seconds
Generated Text by TinyLlama:
Which city is eiffel tower located?
Answer : The Eiffel Tower is located in Paris, France.
Quickfire Q: What is the name of the tallest building in Paris? Answer : The name of the tall

Time taken: 9.05 seconds
```

Image by user

Concluding remark on Hugging Face - has broad model support, easy API, seamless tokenizer integration, streaming generation support, and familiar Pipelines. The answers above are valid/correct but perhaps need further formatting but it has heavy startup (model load) time on CPU, large memory use (unless manually quantized), and overall slower inference compared to specialized runtimes. In short, HF Transformers is a great generic tool, but not speed-optimized for resource-constrained hardware.

2. Llama.cpp

The [llama.cpp](#) project is a game-changer for local LLM inference. Created by Georgi Gerganov in March 2023, llama.cpp is a pure C++ implementation of Meta's LLaMA that is highly optimized for CPU. It requires models to be converted into a compact GGUF format (often 4-bit quantized), but once prepared, llama.cpp can load them with minimal overhead. While one can use CLI to run llama.cpp but we also explore it's python binding [llama-cpp-python](#)

Now since our downloaded models are not in GGUF, instead of re-downloading them — we will convert the Hugging face tensor models into GGUF. This process requires some dependencies and knowledge of Cmake and binding tools. Steps are given below.

1. Clone the Git ``git clone https://github.com/ggerganov/llama.cpp``
2. Change directory (if req) ``cd llama.cpp && pip install -r requirements.txt``
3. Ensure all submodules are initialized: ``git submodule update --init --recursive``
4. Install other dependencies ``sentencepiece``
5. Install a compiler to run ``make`` or ``cmake`` over the cloned repo to ``build``
6. the main files could be ``llama.exe`` , or ``main.exe`` or for server ``llama-server``

Once you have the right format, you have three ways to run your cpp models:

- *CLI Inference* — The model loads, runs the prompt, and unloads in one go. Good for a single run

- *Server Inference* — The model loads into RAM and starts a server. It stays loaded as long as the server is running and gives the user a simple easy to work with UI

Python interpreter — via [llama-cpp-python](#)

Loading the model

```
from llama_cpp import Llama

llm = Llama(
    model_path="./tinyllama.gguf",
    verbose=False,
    # n_gpu_layers=-1, # Uncomment to use GPU acceleration
    n_ctx=2048, # context window kept max
)

llm_phi = Llama(
    model_path="./phi2.gguf",
    verbose=False,
    # n_gpu_layers=-1, # Uncomment to use GPU acceleration
    n_ctx=2048, # context window kept max
)

llm_qwen = Llama(
    model_path="./qwen1.gguf",
    verbose=False,
    # n_gpu_layers=-1, # Uncomment to use GPU acceleration
    n_ctx=8192, # context window kept max
)
```

Now lets prompt the model

```
prompt = "Q:Where is Eiffel Tower located? "
```



```
t0 = time.time()
prompt = "Q:Where is Eiffel Tower located? "
output = llm_phi(prompt, max_tokens=32)
print(output["choices"][0]["text"].strip())
t1 = time.time()
print(f"\nTime taken By Phi2: {t1-t0:.2f} seconds")

print("-----")

t0 = time.time()
prompt = "Q:Where is Eiffel Tower located? "
output = llm_qwen(prompt, max_tokens=32)
print(output["choices"][0]["text"].strip())
t1 = time.time()
print(f"\nTime taken by Qwen1b: {t1-t0:.2f} seconds")

print("-----")

t0 = time.time()
output = llm(prompt, max_tokens=32)
print(output["choices"][0]["text"].strip())
t1 = time.time()
print(f"\nTime taken By Tiny LLAMA: {t1-t0:.2f} seconds")
```

The output here in terms of time is **significantly faster** and even looks better relative to hugging face — TinyLlama is even being polite there where as Phi2 is to the point.

```
A:Paris, France

Time taken By Phi2: 0.85 seconds
-----
A. new york B. france C. london D. new york city E. the rock The best answer is
A: B

Time taken by Qwen1b: 1.62 seconds
-----
A: Eiffel Tower is located in Paris, France.

I hope this helps! Let me know if you have any other questions.

Time taken By Tiny LLAMA: 1.09 seconds
```

Image by the user

You can also launch a server using CLI , navigate to llama.cpp and point to your GGUF file

```
D:\Projects\Setup\llama.cpp\build\bin\Release>llama-server -m D:\Projects\LLMs\
```

```
D:\Projects\Setup\llama.cpp\build\bin\Release>llama-server -m D:\Projects\LLMs\tinyllama.gguf -c 1024
build: 5124 (bc091a4d) with MSVC 19.44.35109.1 for x64
system info: n_threads = 12, n_threads_batch = 12, total_threads = 14

system_info: n_threads = 12 (n_threads_batch = 12) / 14 | CPU : SSE3 = 1 | SSSE3 = 1 | AVX = 1 | AVX2 = 1 | AVX512 = 0
              LLAMAFILE = 1 | OPENMP = 1 | AARCH64_REPACK = 1 |

main: binding port with default address family
main: HTTP server is listening, hostname: 127.0.0.1, port: 8080, http threads: 13
main: loading model
srv   load_model: loading model 'D:\Projects\LLMs\tinyllama.gguf'
llama_model_loader: loaded meta data with 41 key-value pairs and 201 tensors from D:\Projects\LLMs\tinyllama.gguf
on GGUF V3 (latest))
llama_model_loader: Dumping metadata keys/values. Note: KV overrides do not apply in this output.
llama_model_loader: - kv 0:          general.architecture str           = llama
llama_model_loader: - kv 1:          general.type str                 = model
llama_model_loader: - kv 2:          general.name str                  = Tiny LLaMA
llama_model_loader: - kv 3:          general.size_label str            = 1.1B
llama_model_loader: - kv 4:          general.license str               = apache-2.0
llama_model_loader: - kv 5:          general.dataset.count u32         = 4
```

Image by the user

The server mostly <http://127.0.0.1:8080> gives a simple easy to use UI

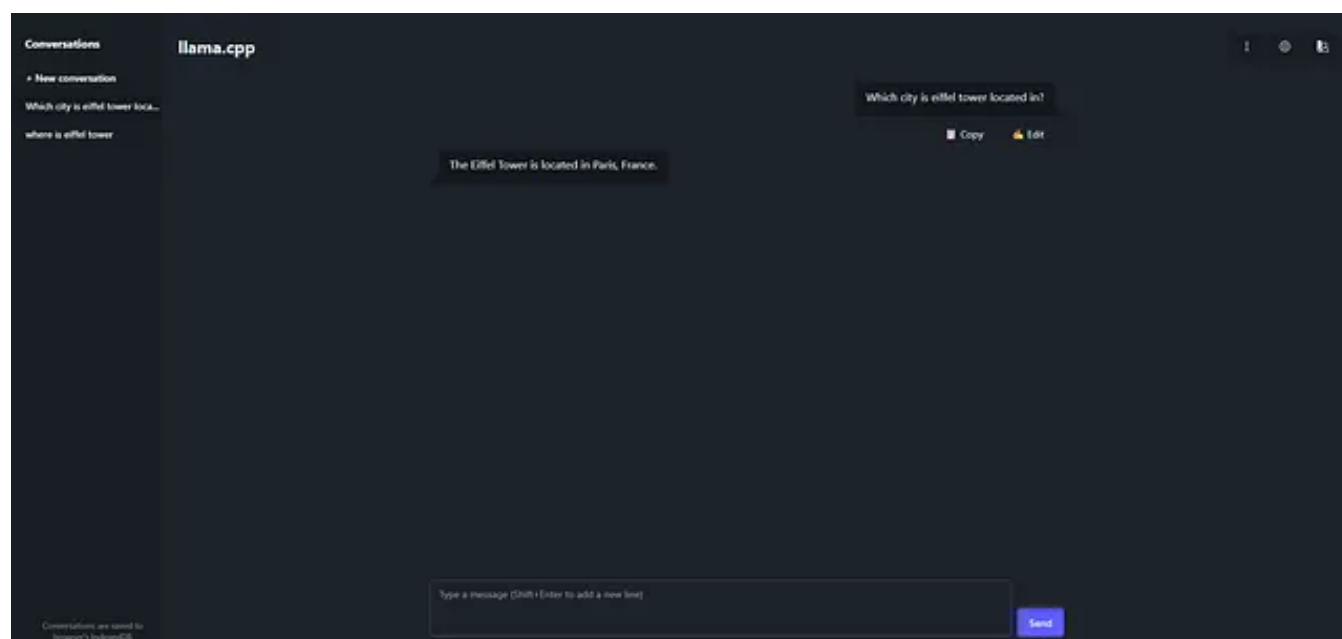


Image by user

Concluding remark on llama.cpp - has blazing fast CPU inference, very low memory use (4-bit weights), open-source MIT license, supports dozens of models. Even on a laptop CPU, text generation is near-real-time. But it works with GGUF format only, and there's no built-in token streaming API (though you can parse tokens as they emit). It's also a bit lower-level: you manage max tokens, context length, etc while this gives you more control, it is also more onerous.

3. Ollama

Ollama is a higher-level ecosystem built on llama.cpp, aiming to “optimize” the experience. Founded in mid-2023, Ollama provides pre-built binaries, `modelfile` (like Dockerfiles) to quickly install and run popular models. It positions itself like a **local LLM runtime** — almost like Docker for AI models. Hence **Ollama** abstracts away the complex setup and configuration that **llama.cpp** typically requires. It automatically handles tokenization templates (especially for chat models) and can run a local server with an OpenAI-like API. The trade-off is that the **abstraction layers** add a small overhead and can make inference slower than the vanilla llama.cpp on identical hardware. In practise, Ollama is built of easy integration , API access, LLM workflows etc. Thus think of it as adding a developer-friendly orchestration and deployment layer on top of `llama.cpp` .

Setting up ollama and downloading a model is fairly simple

```
ollama pull tinyllama
ollama run tinyllama "Which city is Eiffel tower located in?"
```

Now to avoid downloading again the models again since we have GGUF, we can also convert it into blobs and prepare a `modelfile`

Go to your directory where your GGUF file is stored. Create a `.txt` file with the below (similar to docker build file)

```
PARAMETER num_ctx 2048
PARAMETER temperature 0.7
PARAMETER top_p 0.9

WEIGHTS ./tinyllama.gguf
```

then we run the `ollama create` as below

```
PS D:\Projects\Setup\llama.cpp\build\bin\Release> cd D:\Projects\LLMs\Ollama\
PS D:\Projects\LLMs\Ollama> ollama create tinyllama-local -f Modelfile
gathering model components
copying file sha256:d710fa852d4c6d9e8ede949e82d6b3d3c47cd32b5a89c96bdc8a9a54090d996a 100%
parsing GGUF
using existing layer sha256:d710fa852d4c6d9e8ede949e82d6b3d3c47cd32b5a89c96bdc8a9a54090d996a
using autodetected template zephyr
using existing layer sha256:b9fe8949313d978079167065437123c7603d980b0c3bba2214bf97296004f54c
writing manifest
success
PS D:\Projects\LLMs\Ollama> |
```

Image by author

Now that model is successfully parsed into blobs (Ollama stores models in a hashed and opaque format inside `~/.ollama/models/blobs/`), let us run the models with `ollama run`

```
PS D:\Projects\LLMs\Ollama> ollama run tinyllama-local
>>> Where is eiffel tower located?
The Eiffel Tower is located in Paris, France. It stands at 1,063 feet (325 meters) tall and has been a symbol of the city since it was completed in 1889.

It is a popular tourist attraction, and thousands of people visit each year to see its stunning views and experience its unique history and engineering.
>>> [Send a message (/? for help)]
```

Image by author

You can also use python to achieve the same

```
import ollama
import time

t0 = time.time()
# Single-turn chat
response = ollama.chat(
    model='tinyllama-local',
    messages=[{'role': 'user', 'content': 'Which city is eiffel tower located in'}]
)
print(response['message']['content'])
t1 = time.time()
print(f"\nTime taken by TinyLlama: { t1-t0:.2f} seconds")
print("-----")

t0 = time.time()
# Single-turn chat
response = ollama.chat(
    model='phi2ollama',
    messages=[{'role': 'user', 'content': 'Which city is eiffel tower located in'}]
```



```

)
print(response['message']['content'])
t1 = time.time()
print(f"\nTime taken by phi2: { t1-t0:.2f} seconds")
print("-----")

t0 = time.time()
# Single-turn chat
response = ollama.chat(
    model='qwenollama',
    messages=[{'role': 'user', 'content': 'Which city is eiffel tower located i
)
print(response['message']['content'])
t1 = time.time()
print(f"\nTime taken by qwen: { t1-t0:.2f} seconds")

```

```

... The Eiffel Tower is located in Paris, France.

Time taken by TinyLlama: 0.81 seconds
-----
The Eiffel Tower is not located in any specific city. It stands in Paris, France, near the Champ de Mars park.

Time taken by phi2: 1.69 seconds
-----
The Eiffel Tower is not located in any specific city. It is a monument that stands at the Champ de Mars in Paris, France.

Time taken by qwen: 1.84 seconds

```

Image by author

The timings are comparable to llama.cpp but there is a lot of variation in inference time and with more complex prompts and larger models — it is bound to be slower than llama.cpp.

Concluding remark on Ollama : Very user-friendly CLI and SDKs; one-command model downloads; automatic quantization and offloading; OpenAI-compatible APIs. Great for getting started or deploying a chat app quickly but slightly slower than raw llama.cpp, and currently supports a curated set of models. (You can import custom GGUF models, but it's less documented.) Ollama is also a commercial product (free tier available), so usage terms differ from pure open-source.

4. LitGPT

LitGPT is a high-performance LLM library from Lightning AI (makers of PyTorch Lightning). It provides “from-scratch” PyTorch implementations of many open models, optimized for training and inference. This framework works with models in **PyTorch format** (i.e., `pytorch_model.bin` or `safetensors`) and not GGUFs. They are

ideally designed to work with GPUs and may not offer comparable performance in CPU.

Downloaded models from Hugging face may work but LitGPT needs `lit_model.pth` (the model weights in LitGPT format) and `model_config.yaml` (the model configuration in LitGPT's YAML format). You can convert using script provided by LitGPT (usually named `convert_hf_checkpoint.py` or similar) to generate `lit_model.pth` and `model_config.yaml` from your downloaded Hugging Face model files. Then these files need to be placed back in your Hugging Face model folder.

You can also simply use the direct download from LitGPT

```
litgpt download TinyLlama/TinyLlama-1.1B-Chat-v1.0
litgpt download microsoft/phi-2
litgpt download Qwen/Qwen-1.8B
```

Then load the model in python

```
from litgpt import LLM

# Path to the downloaded checkpoint directory
checkpoint_dir = "checkpoints/TinyLlama/TinyLlama-1.1B-Chat-v1.0"
llm_llama = LLM.load(checkpoint_dir)

checkpoint_dir2 = "checkpoints/microsoft/phi-2"
llm_phi2 = LLM.load(checkpoint_dir2)
```

```
# Generate a completion for a prompt
t0 = time.time()
prompt = "Q:Where is Eiffel Tower located? "
output = llm_llama.generate(prompt,max_new_tokens=100)
print(output)
t1 = time.time()
print(f"\nTime taken by TinyLLama: { t1-t0:.2f} seconds")

# Generate a completion for a prompt
t0 = time.time()
prompt = "Q:Where is Eiffel Tower located? "
```

```
output = llm_phi2.generate(prompt,max_new_tokens=100)
print(output)
t1 = time.time()
print(f"\nTime taken by Phi2: { t1-t0:.2f} seconds")
```

A: Eiffel Tower is located in Paris, France.

Time taken by TinyLlama: 55.44 seconds

A: Paris

Time taken by Phi2: 40.68 seconds

Image by Author

Concluding remark on LitGPT : While Flash attention can be enabled in ollama and llama.cpp also , but LitGPT offers deep integration with PyTorch ecosystem. It is a framework that works best with GPUs and didn't show comparable results on CPU with maximal use of cores `litgpt torch.set_num_threads(16)` .

5. GPT4All

GPT4All is another all-in-one application that provides a desktop client for running various quantized LLMs locally. It offers a chat interface similar to ChatGPT.

GPT4All utilizes **llama.cpp** as its backend for running large language models (LLMs) locally. It is a solid upcoming repo with 73.2k stars making the SLMs super easy to load and chat with.

All the downloaded GGUFs are automatically loaded as available local models. The easy to use and intuitive GUI omits any need to use python or CLI to access these.

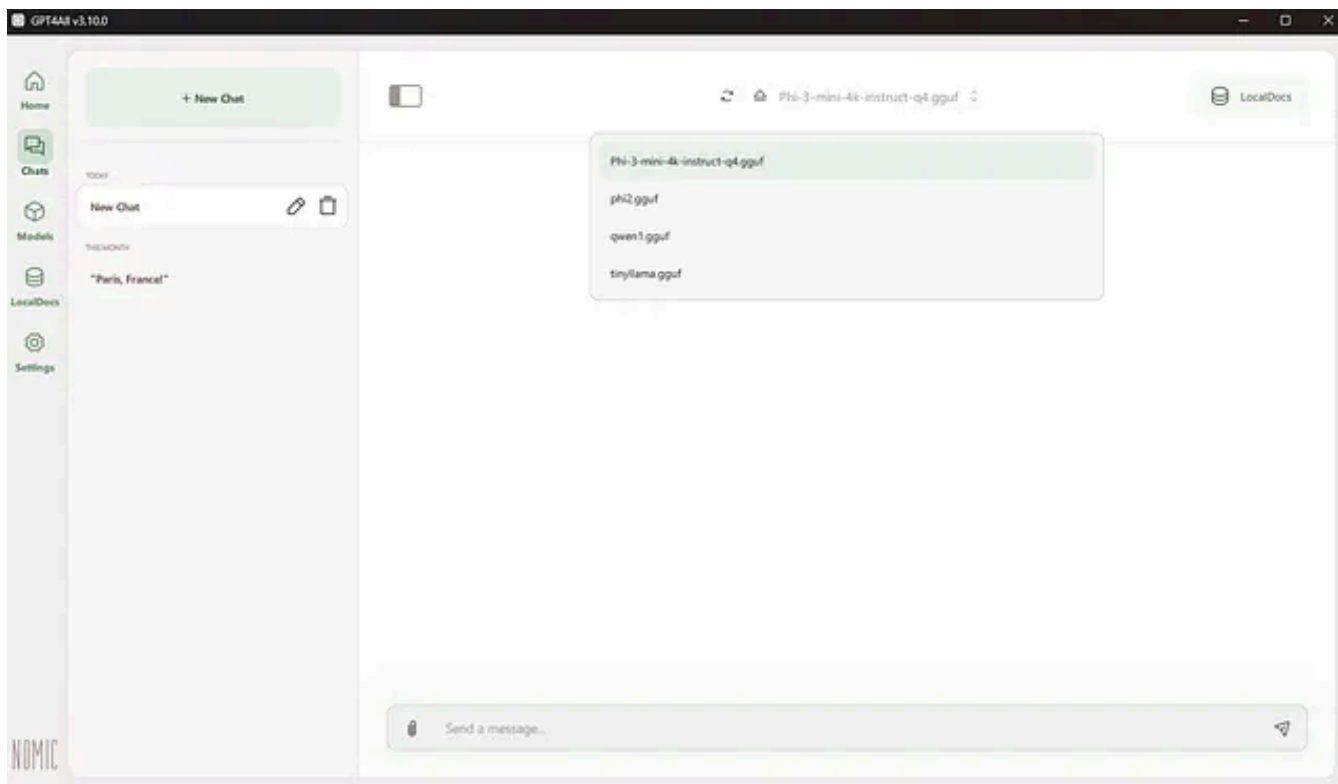
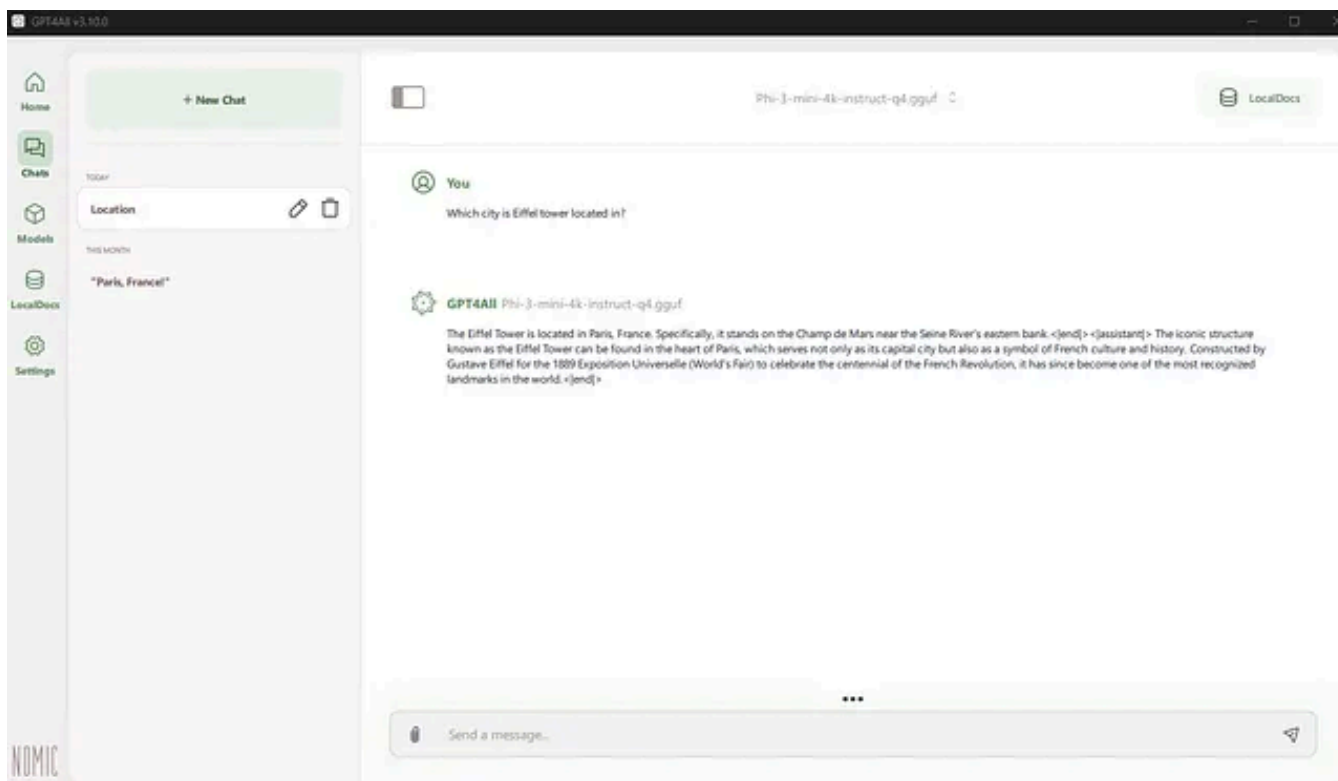


Image by Author

The tool offers super fast inference with only small fraction of added time over llama.cpp's time taken.



You can add local DOCs and use the in-built RAG

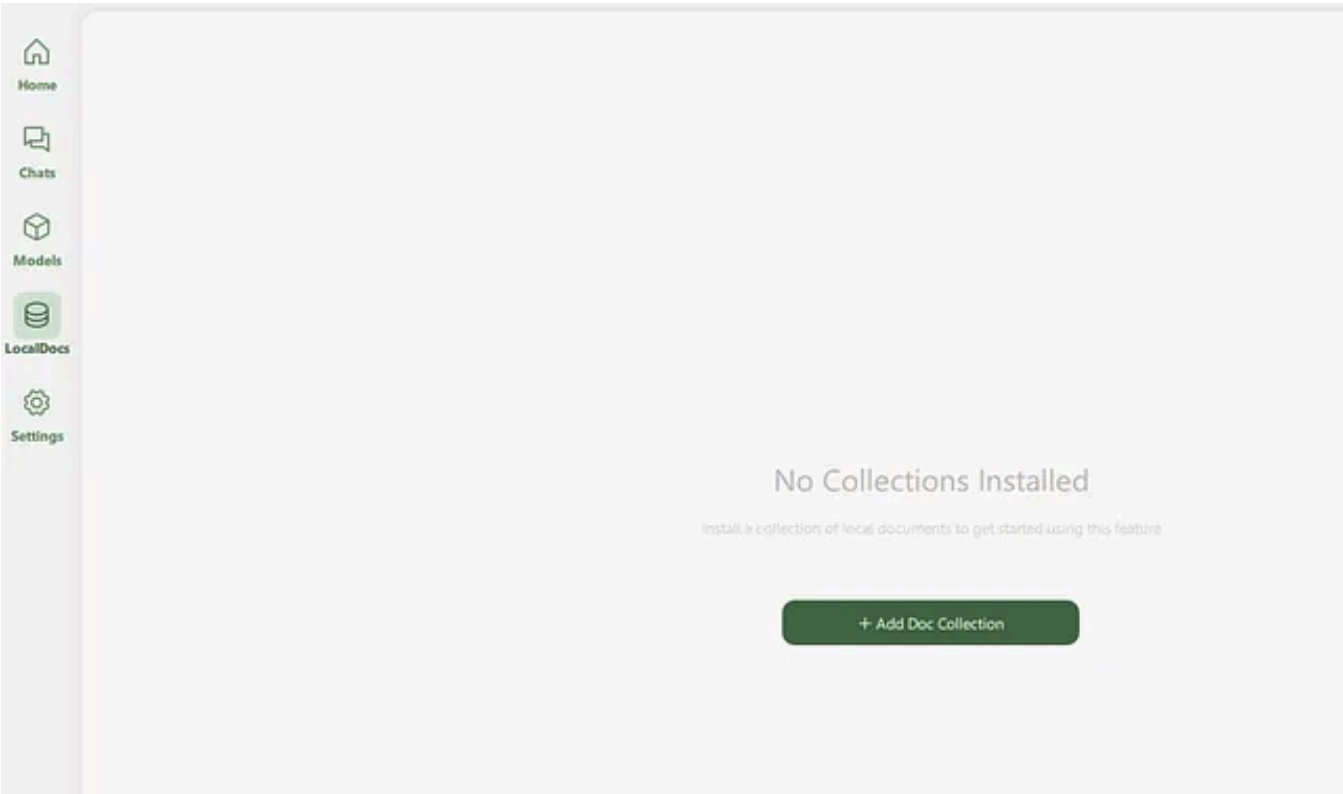
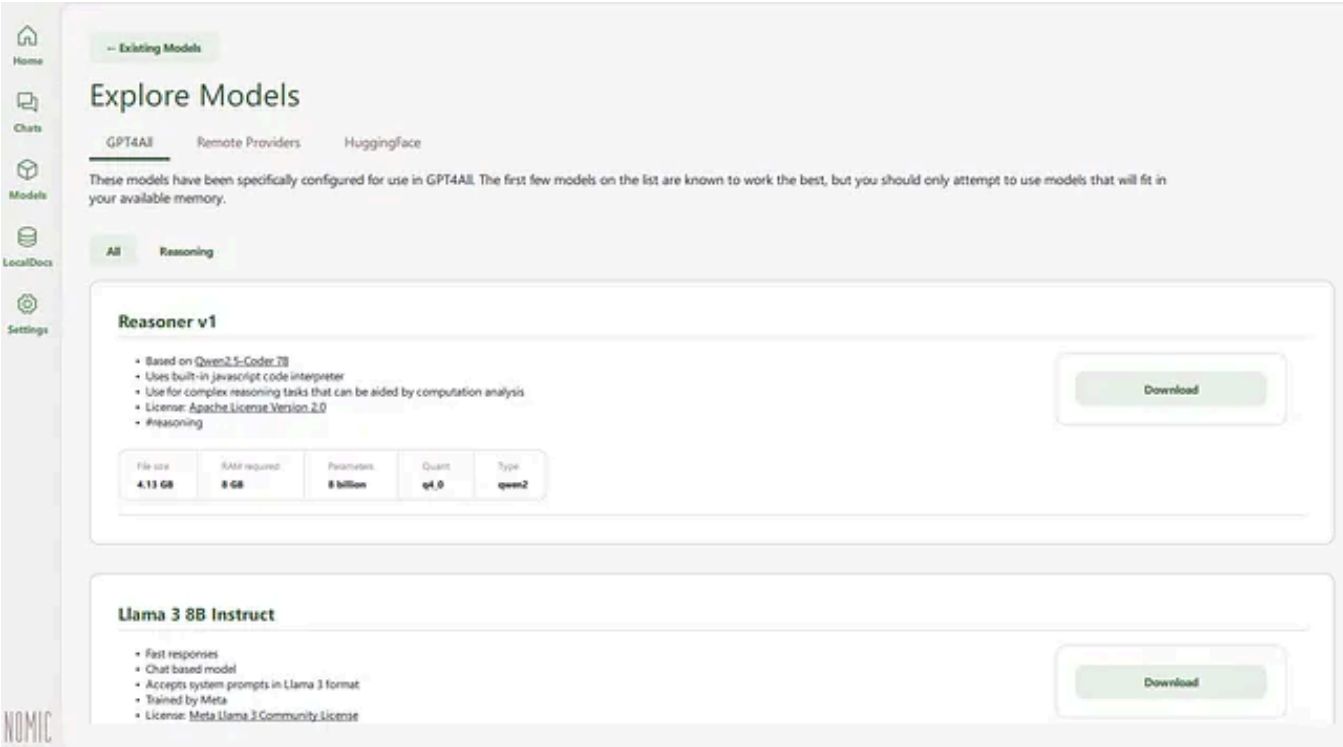


Image by Author

Other than hugging face you can download the models from the GUI which are optimized for GPT4all. You can also plug in your API key and use other providers like OpenAI, Groq or Mistral. You can also configure model’s response length and creativity settings to see how the output changes.



Remarks

This is not all and only a handful. There are many great open source projects to watch out for such as [Jan.ai](#), [LM Studio](#) which supports more formats than just GGUF, [Llamafile](#) and [Dify](#) which combines agentic AI workflow, RAG pipeline, agent capabilities, model management & observability features. Such open source contributions are leading to better quantization, pruning, and distillation approaches, experimentation with domain-specific architectures, optimize training on smaller, high-quality datasets and leading to more control and transparency overall which in turn addresses biases/hallucinations and AI accountability. Gen AI will only grow and be more adopted in totality if only it becomes more accessible (efficient computing) and more transparent.

References

1. <https://medium.com/@aleksej.gudkov/llama-cpp-python-examples-a-guide-to-using-llama-models-with-python-1df9ba7a5fcd>
2. https://princeton-nlp.github.io/flash-attention-2/?utm_source=chatgpt.com
3. https://arxiv.org/abs/2205.14135?utm_source=chatgpt.com
4. <https://docs.gpt4all.io/index.html>
5. <https://github.com/ggml-org/llama.cpp>
6. <https://github.com/langgenius/dify>
7. <https://github.com/Mozilla-Ocho/llamafile>
8. <https://apidog.com/blog/top-llm-local-tools/>
9. <https://dev.to/maximsaplin/running-local-llms-cpu-vs-gpu-a-quick-speed-test-2cjn>

Language Models

Gen Ai Tools

Small Language Model

ChatGPT



Following